

Cinemática Inversa y Validación de Robot SCARA RRP

Santiago Andrés Morales Alarcón-1803552

Jhair Nicolas Muriel Gomez-1803934

Santiago Quintero Paz-1803980

David Antonio Daza Velásquez-1803739

2 de septiembre de 2026

1. Introducción

El presente documento detalla el análisis y la implementación de la cinemática inversa de un robot manipulador de configuración SCARA (Revolute-Revolute-Prismatic). El objetivo es determinar los ángulos articulares y el desplazamiento lineal necesarios para posicionar el efector final en una coordenada espacial (X, Y, Z) deseada.

2. Parámetros Físicos y Geométricos

Basado en el boceto de diseño del laboratorio, las dimensiones físicas de los eslabones son las siguientes:

- Longitud del primer eslabón (L_1): 14.5 cm
- Longitud del segundo eslabón (L_2): 14.0 cm
- Altura de la base al primer eje de rotación (H): 28.0 cm
- Longitud del actuador prismático (l_0): 22.0 cm
- Alcance máximo en el plano XY: 28.5 cm

3. Análisis de Cinemática Inversa

Dado un punto objetivo (X, Y, Z) , el problema se reduce a resolver un sistema geométrico en el plano XY para las dos primeras articulaciones, y una traslación lineal en el eje Z .

3.1. Cálculo de θ_2

Utilizando la ley de los cosenos en el triángulo formado por L_1 , L_2 y la distancia al punto (X, Y) :

$$D = \cos(\theta_2) = \frac{X^2 + Y^2 - L_1^2 - L_2^2}{2L_1L_2} \quad (1)$$

Para obtener los ángulos, empleamos la función `atan2`, la cual arroja dos posibles soluciones (configuración codo arriba y codo abajo):

$$\theta_2 = \text{atan2} \left(\pm \sqrt{1 - D^2}, D \right) \quad (2)$$

3.2. Cálculo de θ_1

Conociendo θ_2 , proyectamos los eslabones sobre los ejes para encontrar θ_1 :

$$\theta_1 = \text{atan2}(Y, X) - \text{atan2}(L_2 \sin(\theta_2), L_1 + L_2 \cos(\theta_2)) \quad (3)$$

3.3. Cálculo de d_3 (Eje Z)

El actuador prismático desciende desde la altura de la base H . El desplazamiento requerido es:

$$d_3 = H - Z \quad (4)$$

4. Implementación en MATLAB y Validación

La validación se realizó mediante un script que calcula la cinemática inversa para un punto arbitrario y posteriormente introduce esos resultados en el modelo de cinemática directa. Si las coordenadas resultantes coinciden con el objetivo inicial, el modelo es matemáticamente correcto.

El uso de la cinemática directa para comprobar el resultado es el siguiente:

$$X_{calc} = L_1 \cos(\theta_1) + L_2 \cos(\theta_1 + \theta_2) \quad (5)$$

$$Y_{calc} = L_1 \sin(\theta_1) + L_2 \sin(\theta_1 + \theta_2) \quad (6)$$

$$Z_{calc} = H - d_3 \quad (7)$$

Al ejecutar el modelo para el punto de prueba ($X = 10, Y = 15, Z = 10$), el algoritmo genera las dos configuraciones válidas de la cinemática inversa y la validación directa confirma que se alcanza el punto con un error de 0,00.

5. Conclusión

El modelo de cinemática inversa desarrollado permite posicionar el efector final del robot SCARA en cualquier coordenada dentro de su espacio de trabajo operativo (radio máximo de 28.5 cm). La comprobación cruzada utilizando cinemática directa garantiza la viabilidad del algoritmo para su futura implementación en los controladores del robot físico.

```

>> inversa_scaras5
--- CINEMÁTICA INVERSA ---
Solución 1 (Codo Abajo): Theta1 = 6.77°, Theta2 = 101.54°, d3 = 18.00 cm
Solución 2 (Codo Arriba): Theta1 = 105.85°, Theta2 = -101.54°, d3 = 18.00 cm

--- VALIDACIÓN (CINEMÁTICA DIRECTA) ---
Posición Calculada: X = 10.00 cm, Y = 15.00 cm, Z = 10.00 cm
>> |

```

Figura 1: resultados matlab SCARA RRP.

6. Anexo: Código en MATLAB

A continuación se presenta el código utilizado para calcular la cinemática inversa y realizar la validación mediante cinemática directa:

```

1  L1 = 14.5; % cm (Primer eslabon)
2  L2 = 14.0; % cm (Segundo eslabon)
3  H = 28.0; % cm (Altura desde el suelo hasta el sistema de
    referencia 1)
4  X_obj = 10;
5  Y_obj = 15;
6  Z_obj = 10;
7
8  [theta1, theta2, d3] = cinematica_inversa_scaras(X_obj, Y_obj, Z_obj
    , L1, L2, H);
9  fprintf('---CINEMÁTICA INVERSA---\n');
10 fprintf('Solución 1 (Codo Abajo): Theta1=%.2f , Theta2=%.2f
    , d3=%.2f cm\n', ...
    rad2deg(theta1(1)), rad2deg(theta2(1)), d3(1));
11 fprintf('Solución 2 (Codo Arriba): Theta1=%.2f , Theta2=%.2f
    f , d3=%.2f cm\n\n', ...
    rad2deg(theta1(2)), rad2deg(theta2(2)), d3(2));
12
13 X_calc = L1*cos(theta1(1)) + L2*cos(theta1(1) + theta2(1));
14 Y_calc = L1*sin(theta1(1)) + L2*sin(theta1(1) + theta2(1));
15 Z_calc = H - d3(1); % El efector baja d3 cent metros desde la
    altura H
16
17 fprintf('---VALIDACIÓN (CINEMÁTICA DIRECTA)---\n');
18 fprintf('Posición Calculada: X=%.2f cm, Y=%.2f cm, Z=%.2f cm
    \n', X_calc, Y_calc, Z_calc);
19
20 function [theta1, theta2, d3] = cinematica_inversa_scaras(x, y, z,
    L1, L2, H)
21     D = (x^2 + y^2 - L1^2 - L2^2) / (2 * L1 * L2);
22     if abs(D) > 1
23         error('El punto objetivo está fuera del alcance máximo de
            %.2f cm.', L1+L2);
24     end
25     theta2_1 = atan2(sqrt(1 - D^2), D);
26     theta2_2 = atan2(-sqrt(1 - D^2), D);
27     theta1_1 = atan2(y, x) - atan2(L2 * sin(theta2_1), L1 + L2 *
        cos(theta2_1));
28     theta1_2 = atan2(y, x) - atan2(L2 * sin(theta2_2), L1 + L2 *
        cos(theta2_2));
29     d3_val = H - z;
30
31     % Retornar arrays
32
33

```

```
34     theta1 = [theta1_1, theta1_2];  
35     theta2 = [theta2_1, theta2_2];  
36     d3 = [d3_val, d3_val];  
37 end
```

Listing 1: Script de cinemática inversa para el SCARA RRP